

PolyAgora V7.4 — Algorithm and Methods

Confidential & Proprietary. This document and the algorithms it describes are the confidential and proprietary information of PolyAgora / PolygonEye. Do not reproduce, distribute, or disclose to any third party without prior written consent.

Working reference for the V7.4 line as actually implemented in this repo. Cross-checked against the governing specs:

- [docs/PolyAgora_Strategy_Manifold_V1.pdf](#) — V7.4 architecture
- [docs/PolyAgora_V7_4b_Soft_Manifold_Upgrade.pdf](#) — V7.4b math
- [docs/Evaluation of V7.4b \(1\).pdf](#) — four-point gate that became V7.4c
- [v74c_validation_outputs/V7.4c_Validation_Note.md](#) — V7.4c passing test pack
- [docs/PolyAgora_V7_5_Spec.md](#) — V7.5 α_1 specification (rejected; superseded by post-mortem)
- [docs/PolyAgora_V7_5_PostMortem.md](#) — V7.5 α_1 rejection + V7.4d adoption rationale

Engine sources:

- [polyagora_v74_engine.py](#) — hard-mask manifold (V7.4)
- [polyagora_v74b_engine.py](#) — soft-manifold engine (V7.4b/V7.4c production)
- [polyagora_v75_engine.py](#) — V7.5 α_1 engine (rejected as α_1); hosts V7.4d at `$\alpha\_M=0$` , `include_mom_eigen=False`
- [polyagora_v63_partner_engine.py](#) — partner data loader, evaluator, baselines, V6.3-gated
- [polyagora_v62_engine.py](#) — Reference Polygon coordinate `(V,T,G,C,R)` and 5-template block probabilities
- [polyagora_v73_engine.py](#) — V7.3 (V6.3 + Q polygons + Driver Seat)
- [run_polyagora_v74_partner.py](#) — orchestrator + signal registry (now includes `v74d`, `v75`, `v75_no_mom_eigen`)
- [build_polyagora.py](#) — canonical build entry point (`--refresh` for Yahoo, `--v75` for V7.5-skinned dashboard)
- [validate_v74c.py](#) — six-test V7.4c validation pack
- [validate_v75.py](#) — six-test V7.5 α_1 validation pack (5/6 pass; Test 4 fails → V7.4d adopted)
- [sweep_v75.py](#) / [analyze_v75_sweep.py](#) — V7.5 α_1 Test 4 parameter sweep + heatmap analyzer

PART A — Architectural overview

A.1 The one-line claim

PolyAgora is a runtime control layer over strategy space, not a strategy. V7.4 turns 14 strategies (the 13 partner-asset eigenfields + a synthetic Momentum 12-1) into eigenfields of the Reference Polygon. At each `t`, the engine 1. measures admissibility `Ai(t)` of each eigenfield, 2. picks (V7.4) or softly aggregates (V7.4b) the most structurally coherent block of eigenfields, 3. caps gross exposure by a zone classifier, 4. routes the synthetic momentum-eigenfield's allocation back through a long-only 12-1 momentum portfolio so the emitted weights stay in the 13-asset partner space.

A.2 Version lineage

```

V6.3 (raw)           $\Sigma \text{ probs\_m} \cdot \text{template\_m}$  — pure regime templates
  |
  ▼ add  $\beta$ -gate
V6.3-gated           $w = \beta \cdot \text{tilt} + (1-\beta) \cdot \text{neutral}$ 
  |
  ▼ add Q polygons × Driver Seat
V7.3                 $\beta' = \beta \cdot Q\_VAIDM \cdot Q\_ADD \cdot Q\_Buffett$ , then same blend
  |
  ▼ replace allocator with eigenfield manifold (hard mask)
V7.4                14 strategy eigenfields, hard Local-Star  $\mathbb{1}\{\phi_i \in B^*\}$ 
  |
  ▼ replace hard mask with soft top-K manifold membership
V7.4b                $M_i(t) = \Sigma_{\{m \in \text{top-K}\}} p_m \cdot \mathbb{1}\{\phi_i \in B_m\}$ 
  |
  ▼ harden zones, lock production configs, pass 6/6 validation tests
V7.4c               v74b code + hardened _classify_zone + plateau + template anchors
  |
  ▼ drop MOM12_1 from  $\Phi$ ; soften block selection ( $\tau \rightarrow 2$ ); same RP
V7.4d               empirical sweep winner from the V7.5 $\alpha_1$  exercise (Sharpe 0.453)
  X
  x V7.5 $\alpha_1$  (Momentum-as-Polygon-Coordinate) — REJECTED. 6D polygon, M in RP.
  x                               Test 4 plateau 6%, optimal  $\alpha_M \approx 0 \rightarrow M$  didn't help.
  x                               See `docs/PolyAgora_V7_5_PostMortem.md`.

```

V7.4c and V7.4d are **not separate engine files** — they are `polyagora_v74b_engine.py` / `polyagora_v75_engine.py` (the V7.5 engine subsumes V7.4b at $\alpha_M=0$) plus locked configurations registered as `v74b_plateau`, `v74b_template`, and `v74d` in `run_polyagora_v74_partner.py`.

A.3 Universe and categories

Universe $\Phi = \mathbf{14 \text{ strategy eigenfields}}$. They are not free-floating signals — each one lives inside the **Reference Polygon** $RP = (V, T, G, C, R)$ and is admissible or inadmissible depending on where the market state X_t sits inside that polygon. The polygon is defined immediately below (§A.4); refer there for the full meaning of each axis.

Category (CATEGORY map)	Members	Eigenfield meaning
<code>persistence</code>	ES, FESX, NKD, MOM12_1	Trend / momentum
<code>carry</code>	TN, FGBL	Yield / stability
<code>defensive</code>	GC, DX	Capital preservation
<code>convexity</code>	SI, BTC	Stress asymmetry
<code>stress</code>	CL, HG, ZS, ZW	Commodity / macro rupture

The 13 partner futures (BTC, CL, DX, ES, FESX, FGBL, GC, HG, NKD, SI, TN, ZS, ZW) are pre-scaled to 10% annual vol by the partner workbook. `MOM12_1` is a synthetic stream built inside the engine — see §A.8 for its construction and dual role.

A.4 Reference Polygon — the (V, T, G, C, R) coordinate

The Reference Polygon is the **fundamental object** of the entire PolyAgora architecture from V6.2 onward. It is the five-dimensional structural space in which the market lives; every other concept in this document (admissibility, blocks, Local Star, zones, Driver Seat) is defined on top of it.

At each time t , the market is represented by a single point

$\$X_t = (V_t, T_t, G_t, C_t, R_t) \in [-1, 1]^5$

inside the polygon. X_t is **exogenous** — none of the 13 partner futures feed it, so the coordinate is a pure market-state reading, free of strategy self-reference.

A.4.1 The five axes

Symbol	Meaning	Raw source	Formula	Sign convention
V	Volatility / risk pressure	VIX level	$\tanh((VIX - 18) / 10)$	$V \rightarrow +1$ as VIX rises ($\approx +1$ at $VIX \geq 50$)
T	Trend / directional persistence	SPY.pct_change(63)	$\tanh(5 \cdot \text{spy_63d})$	$T \rightarrow +1$ on a strong equity uptrend
G	Gold-Copper / macro stress bridge	GLD / CPER ratio	$\tanh(2 \cdot (G/C - 4.5) / 4.5)$	$G \rightarrow +1$ when gold dislocates upward vs copper (flight-to-safety / commodity stress)
C	Carry / credit and yield stability	HYG.pct_change(63)	$\tanh(10 \cdot \text{hyg_63d})$	$C \rightarrow +1$ on healthy high-yield credit
R	Rates / duration and funding stress	TLT.pct_change(63)	$-\tanh(5 \cdot \text{tlt_63d})$	$R \rightarrow +1$ when bonds fall (TLT down $\Rightarrow$ rising rates $\Rightarrow$ funding stress) — note the leading minus sign

The five raw market inputs (columns of the v62 yahoo market CSV) are: **VIX** (CBOE volatility index), **SPY** (S&P 500 ETF), **GLD / CPER** (gold and copper ETFs), **HYG** (high-yield credit ETF), **TLT** (20+yr Treasury ETF).

Why exactly these five: they span the structural axes the architecture cares about — risk appetite (**V**), directional momentum (**T**), commodity / macro rupture (**G**), credit health (**C**), and rates / funding (**R**). The tanh-squashing bounds the polygon to a hypercube; ± 1 corresponds to a market state structurally indistinguishable from a known extreme (VIX 50, deep SPY drawdown, gold-dislocation, credit freeze, bond rout).

A.4.2 Construction and anti-hindsight

`build_exogenous_x` (`polyagora_v62_engine.py:170–189`) assembles the raw five-column panel and then **shift-lags it** by `feature_lag = 1` and dropna's. So the X_t used at row t was measurable strictly *before* t — this is the architecture's first and most important anti-hindsight bound (Test 5 verifies it mechanically).

`_coordinate_from_x` (`polyagora_v74_engine.py:254–265`) applies the tanh transform identically to V6.2's `compute_coordinate` — every PolyAgora version since V6.2 reads X_t the same way, so cross-version comparisons share a common coordinate system.

A.4.3 Where each axis goes downstream

- **All five** feed admissibility: $A_{\text{cat}}(t) = \sigma(c_{\text{cat}} \cdot (V, T, G, C, R))$ — each of the five categories has its own coefficient vector (§B.3), so e.g. `persistence` rewards **T**, dislikes **V/G/R**; `defensive` is admissible when **V** and **R** are high; `convexity` and `stress` are admissible under high **V** and high **G**.
- **|V| and |G|** drive the V7.4c zone-classifier saturation cut: when $\max(|V|, |G|) > 0.60$, the engine sharply attenuates the coherence score (this is the explicit zone-collapse fix vs V7.4b).
- $d_{\partial RP} = \max(|V|, |T|, |G|, |C|, |R|)$ is the ∞ -norm distance to the polygon boundary; the V7.4 spec uses it as a zone-classifier input and the engine records it in diagnostics.

A.4.4 Why the polygon is "fundamental"

Three properties of **RP** make it the load-bearing object of the whole architecture:

1. **It is exogenous.** X_t reads pure market state, not strategy returns. The strategies in Φ are then *admissible or not* inside **RP** — this is what makes PolyAgora a governance layer rather than a strategy.
2. **It is bounded.** Tanh-squashing guarantees $X_t \in [-1, 1]^5$, which lets the engine speak about "distance to boundary" (d_{RP}) and "saturation" ($|V|, |G| > 0.60$) in a stable way regardless of regime.
3. **It is stable across versions.** V6.2 $\rightarrow$ V7.4c all use the same coordinate, so V7.4c is a *governance layer over the same polygon* that V6.2's templates lived on, not a parallel architecture. This is what makes **v74b_template** a clean continuity anchor back to V7.3 (Test 1 of the V7.4c validation pack).

A.5 V7.4 pipeline in plain words

At each t :

1. Read the exogenous market state $X_t = (V, T, G, C, R)$ — defined in §A.4. It is already `shift(feature_lag)`-ed inside `build_exogenous_x`, so the value at row t was measurable before t .
2. For each of the 5 categories, compute $A_{\text{cat}}(t) = \sigma(c_{\text{cat}} \cdot X_t)$. Every strategy in that category inherits this admissibility.
3. Build the runtime survival matrix $W_t = \lambda S_0 + (1-\lambda) R_t$ where R_t is the 60-day rolling pairwise correlation of strategy PnL mapped to $[0,1]$, and S_0 is a category-pair prior. R_t is sliced to $\leq t-1$.
4. Score 4 hand-coded blocks (**TREND**, **CARRY**, **STRESS**, **ROTATION**): $\Gamma_m = \text{min off-diag } W_t \text{ on the block}$, $\mu_m = \text{mean } A \text{ over members}$, $Q_m = \mu_m \cdot \Gamma_m$.
5. **Local Star** $B^*_t = \text{argmax}_m Q_m$.
6. **Zone classifier** (V7.4c-hardened): start with $\text{coh} = \Gamma \cdot \mu$, attenuate by V/G saturation, by realized equal-weight proxy drawdown, and by $|\Delta Q|$. Map to $Z \in \{1,2,3,4\} \rightarrow$ gross multiplier $g(Z) \in \{1.0, 0.6, 0.3, 0.1\}$, then EWM-smooth over time.
7. For each strategy i in Φ : $p_i = A_i \cdot G_{\text{cat}}(\text{Driver Seat}) \cdot \mathbb{1}\{i \in B^*\}$.
8. Normalize and apply zone gate: $w_i = g_{\text{smooth}} \cdot p_i / \sum_j p_j$.
9. **Momentum decomposition**: the weight assigned to **MOM12_1** is removed and redistributed across live partner assets via a long-only positive 12-1 momentum portfolio. Without this step, **MOM12_1** would carry weight inside the engine but be invisible in the emitted asset-level weights panel.
10. Cash = $1 - \sum |w_{\text{real}}|$, enforced by `compute_weights`.

A.6 V7.4b — the soft delta

V7.4 fails because step 7's hard mask $\mathbb{1}\{i \in B^*\}$ is a discrete commitment in a continuous-admissibility architecture. V7.4b's fix is one substitution:

$$M_i(t) = \sum_{m \in \text{top-}K} \rho_m(t) \cdot \mathbb{1}\{\phi_i \in B_m\}, \quad \rho_m = \frac{\exp(\tau Q_m)}{\sum_{\ell \in \text{top-}K} \exp(\tau Q_\ell)}$$

$$p_i(t) = A_i(t) \cdot q_i(t) \cdot M_i(t) \cdot G_i(D_t)$$

Default $K=2$, $\tau=4$, $\lambda=0.6$. Blocks are now five **disjoint category blocks** (per category in §A.3) — Test 3 in the V7.4c validation pack shows this is the only block source with stable partitions across rolling windows. Admissibility $A_i(t)$ and the survival matrix W_t still read the same X_t from §A.4; only the block-membership rule changes.

Top-1 block coherence feeds the zone classifier; singleton blocks substitute μ for Γ (which would otherwise be 1 by convention) so isolated strategies don't masquerade as coherent.

A.7 V7.4c — what changed without changing the math

Three things make V7.4b → V7.4c:

1. **Zone hardening** (in `_classify_zone`, `polyagora_v74_engine.py:284–330`): the zone classifier now attenuates $\text{coh} = \Gamma \cdot \mu$ by - VIX / Gold-Copper saturation: $\max(|V|, |G|) > 0.60$ triggers up to a 10× cut, - Realized proxy drawdown below -3% triggers up to a 10× cut, - $|\Delta Q|$ shocks add a soft cap.

This was the explicit fix for the "zone-collapse" blocker raised in §1.4 of `docs/Evaluation of V7.4b (1).pdf` — the previous classifier left $Z=1$ firing > 95% of the time and the soft aggregator ran at full gross.

1. **Two locked production configurations**, registered in `run_polyagora_v74_partner.py`:

Name	Configuration	Role
<code>v74b_plateau</code>	<code>block_source=category, K=2, $\tau=4$, $\lambda=0.5$</code>	Test 4 plateau-center; production default. Robustness over peak Sharpe.
<code>v74b_template</code>	<code>block_source=template, K=5, gate_mode=beta, q_cfg=on</code>	Continuity-theorem recovery anchor — by construction reproduces V7.3 inside Sharpe tolerance ± 0.02 .

1. **Six-test validation pack** under `v74c_validation_outputs/`. All six pass (`V7.4c_Validation_Note.md`): - **Test 1 — Continuity Theorem**: `v74b_template` matches V7.3 within Sharpe ± 0.02 and MaxDD ± 50 bps. Verified. - **Test 2 — Zone Audit**: V7.4b fires $Z=3/Z=4$ in 4/4 historical stress quarters (vs V7.3 4/5). - **Test 3 — Jaccard Stability**: `category` block source — median Jaccard 1.0, non-trivial share 100%. `graph` source fails. - **Test 4 — Parameter Frontier**: 8/9 slices have a $\geq 30\%$ plateau; plateau-center is the production config. - **Test 5 — No-Look-Ahead Audit**: 7/7 mechanical invariants + 3/3 statistical perturbations clean. - **Test 6 — OOS Holdout 2024–2026**: All four signals (EW, V7.3, plateau, template) have positive OOS Sharpe deltas.

A.8 V7.4d — the empirical sweep winner

V7.4d is the empirical refinement that emerged from the V7.5 α_1 exercise (`docs/PolyAgora_V7_5_Spec.md`, `docs/PolyAgora_V7_5_PostMortem.md`). V7.5 α_1 tested whether Momentum could be promoted into the Reference Polygon as a 6th coordinate; it failed Test 4 (plateau 6%, threshold 30%). But the 216-config sweep surfaced a different winner — a V7.4c configuration with **MOM12_1 dropped from Φ** and **softer block selection** ($\tau=2.0$ instead of 4.0):

Parameter	V7.4c plateau	V7.4d
Universe Φ	14 (13 partner + MOM12_1)	13 (MOM12_1 dropped)
Block source	category	category
K	2	2
τ	4.0	2.0
λ	0.5	0.5
Reference Polygon	5D (V, T, G, C, R)	5D (V, T, G, C, R)
Sharpe (IS)	0.407	0.453
MaxDD (IS)	-11.0%	-10.8%
CAGR (IS)	1.6%	1.8%

Why dropping MOM12_1 helps

When MOM12_1 was an eigenfield in Φ , its synthetic PnL participated in the empirical survival matrix R_t . By construction the synthetic stream is correlated with whichever live partner assets are currently trending — that artificially inflated the persistence block's internal coherence γ and biased top-K selection toward persistence. Removing it leaves the partner correlation matrix unpolluted; persistence wins fewer false-positive top-K cells, diversification rises, and the soft top-K aggregation becomes well-calibrated.

Why $\tau=2$ helps

Lower softmax temperature flattens the block-weight distribution p . At $\tau=4$, ~70% of weight piles on the top-1 block; at $\tau=2$, the split is ~60/40. The Local Star is less dominant; secondary blocks contribute more. This compensates for the V7.4c hardened zone classifier, which already cuts gross exposure aggressively in stress regimes — there's headroom to be less aggressive at the block-selection layer.

Code path

V7.4d is registered as v74d in `run_polyagora_v74_partner.build_signal_registry` (lines 188–192). It's instantiated via the V7.5 engine factory with `momentum_sensitivity=0.0` and `include_mom_eigen=False`:

```
registry["v74d"] = make_v75_signal(
    market, data.realized_pnl,
    drivers=V75DriverConfig(momentum_sensitivity=0.0),
    include_mom_eigen=False,
    top_k=2, tau=2.0, lam=0.5,
)
```

The V7.5 engine subsumes V7.4b at $\alpha_M=0$ — Test 1 of the V7.5 validation pack verifies the admissibility is byte-identical between the two paths. Using the V7.5 engine for V7.4d keeps a single code path; nothing functionally V7.5-specific is active in this configuration.

What this means for the V7.5 program

V7.5 α_1 's "Momentum as Polygon coordinate" thesis is rejected on this universe. The V7.5 program continues, but reoriented: - V7.5 α_2 (per-asset M as multiplicative deformer, spec §13) is also deprecated — it's a smaller version of the same architectural bet. - V7.5 β (external polygons as deformers — VAIDM/ADD/Buffett shaping admissibility / block coherence instead of only β) becomes the next V7.5 increment.

See `docs/PolyAgora_V7_5_PostMortem.md` for the full sweep results, the three findings, and the rationale.

A.9 Momentum 12-1 — the two roles

A.8.1 What "12-1" means

"MOM12_1" is the canonical **12-month-minus-1-month cross-sectional momentum** factor first formalized by Jegadeesh & Titman (1993, *Returns to Buying Winners and Selling Losers*, Journal of Finance). For each asset at each date, compute the cumulative return over the trailing **12 months, excluding the most recent 1 month**, then go long, in proportion, on whichever assets came out positive.

- The 12-month lookback captures medium-term trend persistence — the empirical anomaly Jegadeesh-Titman documented and that has since become one of the most-replicated factors in academic finance.
- The 1-month skip is the standard short-term-reversal hedge: very recent winners tend to mean-revert, so excluding the last month sharpens the persistence signal.

In trading-day units this is `lookback = 252` and `skip = 21` — the constants `MOM_LOOKBACK` and `MOM_SKIP` at `polyagora_v74_engine.py:337-338`. Same construction is used by the asset-level baseline and by the V7.4 eigenfield; only the consumer differs.

A.8.2 The two roles

Momentum appears in **two distinct places** in this codebase:

1. As an **asset-level baseline** (`momentum_signal` in `polyagora_v63_partner_engine.py:234`): 12-1 long-only positive cumulative, normalized to gross 1, fall back to equal-weight when no asset is positive. This is the headline `momentum_12_1` signal in the dashboard.
2. As an **eigenfield $\phi_{\text{MOM}} \in \Phi$** inside V7.4 (`_synth_mom_pnl`, `_momentum_portfolio`): the same long-only 12-1 portfolio is run on the realized panel to produce a synthetic PnL stream that participates in the survival matrix `W_t`, gets a `persistence` category admissibility, and is treated like any other strategy. When V7.4 allocates weight to the eigenfield, that weight is decomposed back to live constituents using the same long-only momentum portfolio.

The architectural payoff (per the V7.4 spec): PolyAgora doesn't *compete* with Momentum — it *governs when Momentum is admissible* (admissibility), *how much exposure it can receive* (Local-Star membership $\times$ zone gate), and *how Momentum coexists with Carry or Defensive* (off-diagonal entries in `W_t`).

A.10 Benchmark comparison

In-sample results from `polyagora_v75_outputs/summary_v74.csv` (2008–2026, partner forward-PnL panel, no leverage, capital-budget envelope `$\sum |w_{\text{real}}| + w_{\text{cash}} = 1$`):

Signal	Sharpe	MaxDD	CAGR	What it is
<code>equal_weight</code>	0.436	-10.6%	1.9%	1/N over live partner assets, gross 1
<code>inverse_vol</code>	0.436	-10.6%	1.9%	1/std (63d) renormalized to gross 1
<code>momentum_12_1</code>	0.521	-14.7%	3.3%	12-1 long-only positive momentum, gross 1
<code>polyagora</code>	0.471	-10.6%	1.9%	V6.3 raw template mixture
<code>polyagora_gated</code>	0.685	-9.7%	2.8%	V6.3 + β -gate
<code>v73</code>	0.707	-9.0%	2.9%	V7.3 = β -gate $\times$ Q polygons $\times$ Driver Seat
<code>v74</code>	0.376	-10.6%	1.2%	Hard-mask manifold — confirms the V7.4b PDF diagnosis
<code>v74b</code>	0.372	-11.2%	1.5%	Soft manifold, default ($K=2$, $\tau=4$, $\lambda=0.6$)
<code>v74b_plateau</code>	0.407	-11.0%	1.6%	V7.4c production default
<code>v74b_template</code>	0.691	-9.2%	2.9%	V7.4c continuity anchor $\equiv$ V7.3
<code>v75</code>	0.344	-10.6%	1.4%	V7.5 α_1 (rejected) — $\Phi=14$, M in RP, $\alpha_M=1$, Option B coeffs
<code>v75_no_mom_eigen</code>	0.352	-10.6%	1.4%	V7.5 α_1 (rejected) — $\Phi=13$, M in RP, $\alpha_M=1$
<code>v74d</code>	0.453	-10.8%	1.8%	V7.5α_1 sweep winner — $\Phi=13$, $K=2$, $\tau=2$, $\lambda=0.5$, $\alpha_M=0$

Reading: - The eigenfield-manifold pipeline (`v74`, `v74b`, `v74b_plateau`) underperforms V7.3 on Sharpe at default settings. V7.4c's purpose was *not* to beat V7.3 on this in-sample window, but to prove the architecture is *operationally sound* (continuity, zones, partitions, no-look-ahead, plateau, OOS) so subsequent increments can sit on top with falsifiable attribution. - `v74b_template` $\equiv$ V7.3 by construction (it bypasses admissibility and `W_t`, falls back to V6.2 template probabilities and the V7.3 β -gate). It's both the regression baseline and the evidence that V7.4b is a strict generalization, not a parallel architecture. - The V7.4b PDF's prediction that hard masking damages Sharpe is verified: `v74` Sharpe 0.376 vs `v74b_template` 0.691 — same data, same gates, same Driver Seat. The only structural difference is hard mask vs soft top-K aggregation. - **`v74d` is the new empirical V7.4-line winner** at +46 bps Sharpe vs `v74b_plateau` and softer MaxDD (-10.8% vs -11.0%). It still trails `v74b_template` / V7.3 by ~ 24 bps Sharpe, so V7.3-equivalent behaviour remains the cross-version ceiling on this in-sample window. - **V7.5 α_1 rejected** — `v75` and `v75_no_mom_eigen` both underperform V7.4c plateau at calibrated $\alpha_M=1.0$. The V7.5 α_1 sweep's positive contribution is `v74d`, not the V7.5 architecture itself.

A.11 The honest limits

From `docs/Evaluation of V7.4b (1).pdf`, the V7.4c validation note, and `docs/PolyAgora_V7_5_PostMortem.md`:

- The V7.4b PDF identifies the real performance ceiling not as the navigation layer (which V7.4c proves is working) but as **within-block alpha** — once a block is selected, the engine still uses `q_i  $\equiv$  1`, so block-level exploitation is currently equal-weight inside the chosen coalition. Sharpe-weighted / Kelly-deformed / convexity-overlay candidates fit under the manifold layer without touching it.
- `block_source=graph` is in the codebase but fails Test 3 — it stays as a research branch with no production role.
- `v74b_plateau` has lower Sharpe than `v74b_template`. The plateau pick was deliberate: Test 4 chose robustness over peak Sharpe, since a single-point Sharpe-max is what got V7.4 into trouble in the first place.

- **V7.5α₁ failed Test 4** (plateau 6%, threshold 30%). The architectural bet of promoting Momentum to a polygon coordinate is not supported on this universe. V7.4d is the empirical win from the sweep, but it's a V7.4-line refinement, not validation of the V7.5α₁ hypothesis. The V7.5 program continues on a different layer — V7.5β (external polygons as deformers, [docs/The Better Architecture \(Your Current Direction\).pdf](#)). V7.5α₂ (per-asset M as multiplicative deformer) is also deprecated since it's structurally the same bet at smaller scale.
- **v74d still trails V7.3 / v74b_template** by ~24 bps Sharpe on this in-sample window. V7.4d is the V7.4-line ceiling on architectural fidelity to the manifold; V7.3-equivalence remains the broader Sharpe ceiling.

PART B — Engineering reference

B.1 Data and timing contract

`polyagora_v63_partner_engine.py:33–112` defines the universe and loader:

- `UNIVERSE = ['BTC', 'CL', 'DX', 'ES', 'FESX', 'FGBL', 'GC', 'HG', 'NKD', 'SI', 'TN', 'ZS', 'ZW']`
- `CASH = 'CASH'` — engine-internal residual capital slot.
- `load_partner_xlsx(path) → PartnerData(realized_pnl, forward_pnl, inception, universe)`.
- Capital-budget envelope (`compute_weights`, lines 128–221): $|w_{real_i}| \in [-1, 1]$ per asset, $w_{cash} \in [0, 1]$, $\sum |w_{real}| + w_{cash} = gross_cap$ (default 1.0).
- `forward_pnl` is hashed before/after the weight loop; any mutation raises.
- NaN handling: pre-inception → zero weight; post-inception NaN today → freeze previous weight; the `|frozen|` consumes the budget.

`compute_weights` invariants (must hold for any `SignalFn`):

```
weights.loc[T] = signal_fn(realized.loc[:T], live_assets_at_T, cfg)
sum(|weights.loc[T, UNIVERSE]|) + weights.loc[T, CASH] == cfg.gross_cap
```

B.2 Reference Polygon coordinate

`build_exogenous_x` in `polyagora_v62_engine.py:170–189` produces the 5-column panel:

Column	Definition
<code>vix</code>	raw VIX level
<code>spy_trend_63d</code>	<code>SPY.pct_change(63)</code>
<code>gold_copper_ratio</code>	<code>GLD / CPER</code>
<code>hyg_trend_63d</code>	<code>HYG.pct_change(63)</code>
<code>tlt_trend_63d</code>	<code>TLT.pct_change(63)</code>

Then `X = X.shift(cfg.feature_lag).dropna()` — `feature_lag=1` is the anti-hindsight bound.

`_coordinate_from_x` (`polyagora_v74_engine.py:254–265`, identical to `compute_coordinate` in v62):

```
V = tanh((vix - 18) / 10)
T = tanh(5 * spy_trend_63d)
G = tanh(2 * (gold_copper_ratio - 4.5) / 4.5)
C = tanh(10 * hyg_trend_63d)
R = -tanh(5 * tlt_trend_63d)
```

All five components live in $[-1, 1]$. $d_{\partial RP} = \max(|V|, |T|, |G|, |C|, |R|)$ is the ∞ -norm distance to the polygon boundary.

B.3 Admissibility $A_i(t)$

`polyagora_v74_engine.py:95–102` — admissibility coefficients per category:

```
ADMISSIBILITY_COEFS = {
    #           V       T       G       C       R
    "persistence": [-1.5, +2.5, -0.5, +1.0, -0.5],
    "carry":        [-1.0, +0.5, -0.5, +1.5, -2.0],
    "defensive":     [+1.0, -0.5, +0.5, -0.5, +1.0],
    "convexity":     [+2.0, -0.5, +1.5, -0.5, +0.5],
    "stress":        [+1.0, -0.5, +2.0, -0.5, +0.5],
}
```

Per `_admissibility (:268–274)`: $A_{cat} = \sigma(c_{cat} \cdot X_t)$ is computed once per category, then broadcast to every strategy in that category. So $A_{ES} = A_{FESX} = A_{NKD} = A_{MOM12_1} = A_{persistence(t)}$.

Sign conventions: - **Persistence**: rewards trend T , dislikes vol V , gold dislocation G , rates stress R . - **Carry**: rewards credit health C , dislikes rates stress R . - **Defensive**: admissible under high V and high R (flight-to-safety state). - **Convexity / Stress**: admissible under high V and high G (Gold-Copper dislocation).

B.4 Survival matrix W_t

`polyagora_v74_engine.py:156–217`.

B.4.1 Structural prior S_0 (`_CAT_PRIOR`, `:156–176`)

Symmetric, indexed by category-pair. Diagonal 1.0. Off-diagonals:

Pair	S_0
persistence-persistence, carry-carry, defensive-defensive	0.90
convexity-convexity	0.85
stress-stress	0.80
carry-persistence	0.70
convexity-defensive, defensive-convexity	0.70
carry-defensive	0.65
convexity-stress	0.65
defensive-stress	0.55
convexity-persistence	0.50
defensive-persistence	0.40
carry-convexity	0.40
persistence-stress	0.35
carry-stress	0.30
any other pair (none in practice)	0.50

B.4.2 Empirical R_t (`_empirical_survival` , :197–217)

```
panel = history_lag.tail(window).copy()
panel[MOM_NAME] = mom_pnl_lag # synthetic stream stitched in
corr = panel[STRATEGIES].corr().fillna(0.0)
R = ((corr + 1) / 2).clip(0, 1)
np.fill_diagonal(R, 1.0)
```

- `window = 60` (engine factory default).
- If `len(history_lag) < window/2`, return a flat 0.5 matrix (uninformative).
- Slices `history.iloc[:-1]` — the row at `t` itself is excluded.

B.4.3 Blend

$W_t = \lambda S_0 + (1-\lambda) R_t$, default $\lambda = 0.6$. V7.4c production default reduces λ to 0.5 (`v74b_plateau`, in `run_polyagora_v74_partner.py:175`).

B.5 Blocks

B.5.1 V7.4 hard blocks (`BLOCKS` , `polyagora_v74_engine.py:106–111`)

```
BLOCKS = {
    "TREND": ["ES", "FESX", "NKD", "MOM12_1", "TN", "FGBL"],
    "CARRY": ["TN", "FGBL", "GC", "DX", "ES", "FESX"],
    "STRESS": ["GC", "DX", "SI", "BTC", "CL", "HG", "ZS", "ZW"],
    "ROTATION": ["SI", "BTC", "CL", "HG", "ZS", "ZW", "ES", "FESX", "NKD", "MOM12_1"],
}
```

Overlapping; "STRESS" is the only block without persistence members.

B.5.2 V7.4b block sources (`make_v74b_signal` , `polyagora_v74b_engine.py:375–443`)

block_source	Construction	Status
<code>category</code>	5 disjoint category blocks (<code>CATEGORY_BLOCKS</code> , :160–167).	Production default.
<code>graph</code>	Connected components of (Φ, E_t) where $(i,j) \in E \Leftrightarrow W_t(i,j) \geq \theta_S$ (<code>_connected_components</code> , :108–134).	Research branch — fails Test 3 Jaccard stability.
<code>static</code>	V7.4's 4 hand-coded overlapping blocks.	Regression baseline.
<code>template</code>	V6.2 5-template blocks (A, B, C, D, G); bypasses admissibility and W_t .	Continuity anchor — equivalent to V7.3 at $(K=5, \beta\text{-gate}, Q \text{ on})$.

B.5.3 Block scoring

`_block_internal_survival` (`polyagora_v74_engine.py:220–227`):

```
 $\Gamma_m = \min$  over off-diagonal  $W_t$  entries on members
```

Singletons return 1.0 (zero off-diagonal entries).

`_block_membership` (:230–232):

```
 $\mu_m = \text{mean}(A_i \text{ for } i \text{ in members})$ 
```

$Q_m = \mu_m \cdot \Gamma_m$.

B.6 Soft top-K aggregation (V7.4b)

`_soft_membership_from_blocks` (`polyagora_v74b_engine.py:174–225`):

```
scored = [(members,  $\mu$ ,  $\Gamma$ ,  $\Gamma$ ,  $\mu$ ) for members in blocks]
scored.sort(key=lambda r: r[1], reverse=True)
K = max(1, min(top_k, len(scored)))
top = scored[:K]

Q = [r[1] for r in top]
z =  $\tau$  * Q - max( $\tau$  * Q) # softmax-stability shift
p = exp(z); p /= p.sum()

M[s] = sum(p_m * 1{s in members_m} for (members, _, _, _), p_m in zip(top, p))
```

Defaults: $K = 2$, $\tau = 4$. The V7.4b spec's recommended sweep is $K \in \{2, 3\}$, $\tau \in \{2, 4, 6\}$.

Then $p_i = A_i \cdot G_{\text{cat}}(D_t) \cdot M_i$ (with $q_i \equiv 1$), $w_{\text{raw}} = g_{\text{smooth}} \cdot p / \sum p$.

B.7 Zone classifier — V7.4c hardened

`_classify_zone` (`polyagora_v74_engine.py:284–330`):

```
s = max(1e-3, drivers.boundary_sensitivity)
V_, G_ = abs(coord[0]), abs(coord[2])
stress_axis = max(V_, G_)

coh =  $\gamma$  *  $\mu$ 

# V/G saturation
if stress_axis > 0.60:
    coh *= max(0.10, 1.0 - (stress_axis - 0.60) * 2.25)

# Realized drawdown
if proxy_dd < -0.03:
    coh *= max(0.10, 1.0 - (-proxy_dd - 0.03) * 8.0)

#  $\Delta Q$  flux dampener
coh *= (1.0 - 0.30 * min(1.0, dQ * 5.0))

# Thresholds – sliding with boundary_sensitivity
if coh >= 0.35 / s: return 1
if coh >= 0.22 / s: return 2
if coh >= 0.12 / s: return 3
return 4
```

Gross multiplier: `ZONE_GROSS_BASE = {1: 1.00, 2: 0.60, 3: 0.30, 4: 0.10}`.

EWM smoothing: $\alpha = 1 - \exp(-\ln 2 / \text{halflife})$, `halflife = 5.0 / recovery_aggression`, applied to `g_raw` on each tick (`polyagora_v74_engine.py:472–475`).

`proxy_dd` is the equal-weight live-universe drawdown at `t-1` (`_proxy_drawdown` in `polyagora_v63_partner_engine.py:429–433`, sliced `loc[:t].iloc[:-1]` in the V7.4 engine).

$dQ = |Q_t - Q_{t-1}|$ carried through `state["last_Q"]` (or `state["last_Q1"]` for V7.4b, using only the top-1 block).

B.8 V7.4b zone-classifier wrinkle for singletons

`polyagora_v74b_engine.py:505–507`:

```

Q1      = top[0]["Q"]
gamma1  = top[0]["gamma"] if top[0]["size"] > 1 else top[0]["mu"]
mu1     = top[0]["mu"]
Z = _classify_zone(gamma1, mu1, coord, dQ, drivers, proxy_dd=proxy_dd)

```

The rationale (paraphrasing the comment): a singleton block has $\Gamma \equiv 1$ (no off-diagonal entries), which would over-promote it in the zone classifier. Collapsing Γ to μ for singletons prevents an isolated strategy from masquerading as fully coherent.

This matters whenever `block_source` $\in$ {graph, static} produces singletons; with `category` (the default), all five blocks have size ≥ 2 so this branch is rarely hit.

B.9 Driver Seat

`V74DriverConfig` (`polyagora_v74_engine.py:118–136`):

Dial	Default	Effect
<code>convexity_preference</code>	0.0	Multiplier on <code>convexity</code> and <code>stress</code> category scores; <code>G_cat = max(0, 1 + κ)</code>
<code>carry_preference</code>	0.0	Same on <code>carry</code>
<code>defensive_preference</code>	0.0	Same on <code>defensive</code>
<code>boundary_sensitivity</code>	1.0	Divides zone thresholds — $> 1 \Rightarrow$ tighter $\Rightarrow$ drops to lower Z sooner
<code>recovery_aggression</code>	1.0	Divides EWM half-life on <code>g(Z_t)</code> — $> 1 \Rightarrow$ faster reflation

Five named presets in `run_polyagora_v74_partner.py:62–69` :

Preset	Setting
<code>default</code>	All defaults — pure-spec manifold
<code>defensive</code>	<code>boundary_sensitivity=1.5, defensive_preference=0.5</code>
<code>carry</code>	<code>carry_preference=0.5</code>
<code>convex</code>	<code>convexity_preference=0.5</code>
<code>active</code>	<code>recovery_aggression=2.0</code>

`V74bDriverConfig` is `V74DriverConfig` verbatim — the soft-manifold parameters `K`, `$\tau$` , `$\lambda$` , `$\theta_S$` are engine-factory args, not Driver Seat dials.

B.10 Synthetic MOM12_1 and decomposition

`_synth_mom_pnl` (`polyagora_v74_engine.py:364–380`) — vectorized PnL stream:

```

cum      = realized.rolling(window=252, min_periods=126).sum()
signal   = cum.shift(skip + 1)           # skip = 21
pos      = signal.clip(lower=0.0).fillna(0.0)
gross    = pos.sum(axis=1)
weights  = pos.div(gross.replace(0.0, NaN), axis=0).fillna(0.0)
pnl      = (weights * realized.fillna(0.0)).sum(axis=1)

```

Anti-hindsight: the position at `t` uses `cum.shift(skip+1)`, so it depends only on `realized.iloc[:t-skip-1]` ($\leq t-22$). This stream feeds the survival matrix as if it were a 14th asset.

`_momentum_portfolio` (:341–361) is the **decomposition** portfolio applied at runtime — same construction, but on `history` only (the function caller does its own `iloc[:-1]` slicing where needed):

```

window = history.iloc[-(252+21):-21]      # rows ≤ t-22
cum     = window[live].sum(min_count=126)
raw     = cum.clip(lower=0).fillna(0)
return raw / raw.abs().sum()              # or 1/N fallback

```

The decomposition step (`polyagora_v74_engine.py:498–505`):

```

w_mom = w_strategy.pop(MOM_NAME, 0.0)
mom_port = _momentum_portfolio(history, live)
for a in live:
    w_strategy[a] += w_mom * mom_port.get(a, 0.0)

```

This is also captured per-asset in diagnostics as `mom_<asset>` (saved as `weights_<signal>_mom.csv` by `run_signal`) — that's the dashboard's "internal view" of where the MOM12_1 eigenfield's allocation ended up.

B.11 Anti-hindsight invariants

Eight invariants the engine asserts, mechanically (Test 5 checks 7/7 of these in source):

1. `X_t` is `shift(feature_lag)` -ed inside `build_exogenous_x`.
2. `R_t` reads `history.iloc[:-1]` only.
3. The synthetic MOM stream uses `cum.shift(skip+1)`, so row `t` depends on rows $\leq t\text{-skip}-1 = t-22$.
4. `_momentum_portfolio` slices `window = history.iloc[-(L+s):-s]`, excluding the most recent `s = 21` rows.
5. `proxy_dd` at `t` reads `proxy_dd_full.loc[:t].iloc[:-1]`.
6. `compute_weights` slices `realized.loc[:t]` and asserts `history.index[-1] == t`.
7. `_compute_beta_from_market` ends with `.shift(1)` (used by `v74b_template`).
8. `forward_pnl` hash is verified equal before/after the weight loop — any mutation raises.

B.12 Equal-weight, inverse-vol, momentum_12_1 baselines

Defined in `polyagora_v63_partner_engine.py:228–263`:

```

def equal_weight_signal(history, live, cfg):
    return pd.Series(1.0, index=live)      # engine renorms to gross 1

def momentum_signal(history, live, cfg):    # 12-1, long-only positive
    if len(history) < L + s: return Series(1.0, index=live)
    window = history.iloc[-(L+s):-s]
    cum = window[live].sum(min_count=L//2).fillna(0.0)
    raw = cum.clip(lower=0.0)
    return raw / raw.abs().sum() if positive else Series(1.0, index=live)

def inverse_vol_signal(history, live, cfg):
    if len(history) < cfg.vol_lookback: return Series(1.0, index=live)
    vol = history[live].iloc[-63:].std()
    return (1.0 / vol.replace(0, NaN)).fillna(0.0)

```

All three are pre-normalized so the engine's gross-budget renorm passes them through unchanged. None of them ever see `forward_pnl`.

The dashboard label and color mapping is in `run_polyagora_v74_partner.py:78–114`.

B.13 Evaluation contract

`evaluate` (`polyagora_v63_partner_engine.py:547–557`):

```
contrib = weights[real].values * forward_pnl[real].fillna(0.0).values
rets = pd.Series(contrib.sum(axis=1), index=forward_pnl.index)
equity = (1 + rets).cumprod()
```

- **CASH** contributes zero by construction.
- **forward_pnl** NaN on an asset is treated as zero contribution (asset didn't earn that day).
- Sharpe in **summary_row** is `mean(rets) / std(rets) * sqrt(252)`.

B.14 Where to look when something looks wrong

Symptom	First file to read
Gross > 1 on a row	<code>polyagora_v63_partner_engine.py:194–211</code> (<code>compute_weights</code> budget enforcement)
Zone stuck at 1	<code>polyagora_v74_engine.py:284–330</code> (<code>_classify_zone</code>); confirm <code>proxy_dd</code> and <code>stress_axis</code> are passing through
Local Star never changes	<code>polyagora_v74_engine.py:106–111</code> (block definitions); for V7.4b check <code>block_source</code>
Sharpe of <code>v74b_template</code> ≠ V7.3 by > 0.02	<code>polyagora_v74b_engine.py:250–368</code> (template branch); confirm <code>top_k=5</code> , <code>gate_mode="beta"</code> , <code>q_cfg</code> matches V7.3 defaults
MOM12_1 weight not reflected in asset weights	<code>polyagora_v74_engine.py:498–505</code> (decomposition step); per-asset captured in <code>weights_<signal>_mom.csv</code>
Diagnostics empty	<code>_signal.diagnostics_df()</code> is rebuilt each call; only populated after at least one tick

B.15 The V7.5 engine and V7.4d

V7.5α₁'s engine lives in `polyagora_v75_engine.py`. While the V7.5α₁ architectural bet was rejected (`docs/PolyAgora_V7_5_PostMortem.md`), the engine remains the canonical V7.4-line code path because:

- It **subsumes V7.4b** at `α_M = 0`: `make_v75_signal(momentum_sensitivity=0.0, include_mom_eigen=True)` produces byte-identical admissibility values to `make_v74b_signal` at matching `(K, τ, λ)`. V7.5 Test 1 verifies this mechanically.
- It supports `include_mom_eigen=False` — the universe-toggling switch that turned out to matter empirically (V7.4d).
- It exposes a `momentum_sensitivity` dial that future M-related experiments can reuse without re-implementing the universe / coefficient plumbing.

V7.4d is registered as `v74d` in `run_polyagora_v74_partner.build_signal_registry` (lines 188–194):

```
registry["v74d"] = make_v75_signal(
    market, data.realized_pnl,
    drivers=V75DriverConfig(momentum_sensitivity=0.0),
    include_mom_eigen=False,
    top_k=2, tau=2.0, lam=0.5,
)
```

Helper functions added in `polyagora_v75_engine.py` and reused by V7.4d:

- `_build_universe(include_mom_eigen)` — returns `(strategies, category, category_blocks, S_prior)`, dropping `MOM12_1` from all four when False.

- `_empirical_survival_v75(history_lag, mom_pnl_lag, window, strategies)` — parameterized rolling correlation that handles both $\Phi=13$ and $\Phi=14$ universes.
- `_soft_membership_v75(adm, W, blocks, *, top_k, tau, strategies)` — V7.4b's soft top-K parameterized over an arbitrary strategy list.
- `build_m_series(realized, ...)` — Momentum Persistence series (unused by V7.4d but present in the engine).

B.16 Open work

1. **Within-block alpha audit:** `q_i  $\equiv$  1` throughout V7.4/V7.4b/V7.4c/V7.4d. The V7.4b evaluation PDF's diagnosis — "the engine is very good at not dying, increasingly good at steering, but moderate at extracting convexity" — points to within-block exploitation as the real ceiling. Candidates: Sharpe-weighted, Kelly-deformed, convexity overlay.
2. **V7.5 β** (next V7.5 increment): external polygons (VAIDM / ADD / Buffett) generalized from V7.3-style β -gate multipliers to *admissibility / block-coherence deformers* per `docs/The Better Architecture (Your Current Direction).pdf`. Each layer ships with the same six-test pack as a gate.
3. **V7.4d OOS validation:** Test 6 (OOS holdout 2024-2026) was run on V7.5 α_1 variants only. Re-run the same split on V7.4d to confirm the +46 bps IS Sharpe holds OOS — it should, since V7.4d's mechanism (cleaner `R_t` without MOM12_1's synthetic stream) is structural rather than parameter-fragile.
4. **block_source = clique or community**: V7.4b spec §1.8 alternatives to connected components. Not implemented.
5. **V7.5 α_1 artifacts:** `polyagora_v75_engine.py`, `validate_v75.py`, `sweep_v75.py`, `analyze_v75_sweep.py`, `docs/PolyAgora_V7_5_Spec.md`, `docs/PolyAgora_V7_5_PostMortem.md`, `v75_validation_outputs/` remain in the repo as the V7.5 α_1 falsification record. Do not delete — the validation pack will be reused by V7.5 β with minimal modification.